



ML FOR TEXT MINING

Section 1

Case Study 1: Sentiment Analysis of Two Reviews

- We have two customer reviews:

Review 1: "The product is amazing! It works perfectly and I love it."

Review 2: "This product is terrible. It broke after one use, and I hate it!"

Our goal is to determine whether each review is **Positive, Negative, or Neutral** using step-by-step text mining techniques.

Case Study 1: Sentiment Analysis of Two Reviews

Step 1: Preprocess & Clean Text

1. Tokenization: We break each sentence into words (tokens).

Review 1: ["The", "product", "is", "amazing", "!", "It", "works", "perfectly", "and", "I", "love", "it", "."]

Review 2: ["This", "product", "is", "terrible", ".", "It", "broke", "after", "one", "use", "and", "I", "hate", "it", "!"]

2. Removing Stopwords and Punctuation: Stopwords do not carry sentiment, so we remove them.

Review 1: ["product", "amazing", "works", "perfectly", "love"]

Review 2: ["product", "terrible", "broke", "one", "use", "hate"]

3. Stemming/Lemmatization: Convert words to their root forms.

Review 1: ["product", "amaze", "work", "perfect", "love"]

Review 2: ["product", "terrible", "break", "one", "use", "hate"]

Case Study 1: Sentiment Analysis of Two Reviews

Step 2: Define Sentiment Dictionary

A Sentiment Dictionary, or **Lexicon**, is a curated database where words are mapped to numerical values representing their **Polarity** (positive or negative) and sometimes their **Intensity**.

In this step, we assign a **weight** to each root word. For example, "amaze" is assigned **+1** (Positive Polarity), while "terrible" is assigned **-1** (Negative Polarity).

```
# Define Sentiment Dictionary (Score Dictionary)
lexicon = {
    "amaze": 1, "perfectly": 1, "love": 1,
    "terrible": -1, "break": -1, "hate": -1
}
```

Case Study 1: Sentiment Analysis of Two Reviews

Step 3: Compute Sentiment Score

This is the quantitative phase where individual word values are combined to create a single Compound Score representing the overall sentiment of the text.

$$\text{Total Sentiment} = \text{Sum}(\text{Polarity of each word})$$

- **Review 1:** 1 (amaze) + 1 (perfectly) + 1 (love) = +3
- **Review 2:** -1 (terrible) + -1 (break) + -1 (hate) = -3

The resulting number indicates both the **direction** and **strength** of the sentiment.

Example: A review with three negative words will result in a score of -3. This tells that the review is not just negative, but *intensely* negative compared to a review with a score of -1.

Case Study 1: Sentiment Analysis of Two Reviews

Step 4: Assign Sentiment

The system uses a set of rules to categorize the total score. Usually, the "Zero-Point" is used as the neutral boundary.

Sentiment Label	Numerical Condition
Positive	Score > 0
Neutral	Score = 0
Negative	Score < 0

- **Review 1 Classification:** Since the score is **+3** (above zero), the logic assigns a **Positive** label.
- **Review 2 Classification:** Since the score is **-3** (below zero), the logic assigns a **Negative** label.

Case Study 2: Sentiment Analysis with Polarity Flipping

Negation words like **"not," "never,"** and **"no"** act as "switches" that flip the sentiment of the very next word.

The review **"Not a great experience, but it works"** might be labeled positive because it contains two favorable words ("great" and "works") against only one negative word ("not").This approach fails because it treats words as independent units.

Instead of treating "not" as a standalone negative word, we look ahead. By "looking ahead," the phrase "not great" is transformed from a positive value to a negative one.

Category	Original Count	After Negation Handling
Positive	2 ("great", "works")	1 ("works")
Negative	1 ("not")	2 ("not great")

Case Study 3: Topic Modeling of Docs

Modern Generative AI aims to capture **hidden patterns** within complex information. To do this, it creates a **Latent Space**. Instead of seeing data as random bits, the AI organizes concepts by their underlying meaning.

In the world of AI, **text is a data distribution**. Words are not random; they follow mathematical patterns of probability.

Topic Modeling is the bridge that allows us to understand these distributions. It is the tool used to **extract the latent descriptions** (the topics) from the raw data distribution.

Topic Modeling acts as the 'Discovery' tool that extracts **latent descriptions** from existing data, providing the thematic map that Generative AI then uses to navigate and create new content.

Case Study 3: Topic Modeling of Docs

Topic is a recurring theme or concept hidden within your text. In data science, a topic is not just a title; it is a mathematical cluster of related words—like "water," "ocean," and "rain" grouped together.

Topic Modeling is the unsupervised Machine Learning technique that scans thousands of documents to detect these word patterns and reveal these "latent" (hidden) themes without any manual tagging.

It identifies the main subjects and maps out which documents belong to which subject.

Case Study 3: Topic Modeling of Docs

- We have some Documents:

Doc 1: "A whopping 96.5 percent of water on Earth is in our oceans, covering 71 percent of the surface of our planet. And at any given time, about 0.001 percent is floating above us in the atmosphere. If all of that water fell as rain at once, the whole planet would get about 1 inch of rain."

Doc 2: "One-third of your life is spent sleeping. Sleeping 7-9 hours each night should help your body heal itself, activate the immune system, and give your heart a break. Beyond that--sleep experts are still trying to learn more about what happens once we fall asleep."

Doc 3: "A newborn baby is 78 percent water. Adults are 55-60 percent water. Water is involved in just about everything our body does."

Doc 4: "While still in high school, a student went 264.4 hours without sleep, for which he won first place in the 10th Annual Great San Diego Science Fair in 1964."

Doc 5: "We experience water in all three states: solid ice, liquid water, and gas water vapor."

Our goal is to identify the most prevalent hidden themes within the documents to understand the core subjects that define this specific collection of text.

Case Study 3: Topic Modeling of Docs

Step 1: Preprocess & Clean Text

1. **Lowercasing:** Standardizes words.
2. **Stopword Removal:** Deletes words that carry no thematic meaning.
3. **Punctuation Removal:** Cleans out commas, periods, and hyphens.
4. **Lemmatization:** Reduces words to their root form.

Case Study 3: Topic Modeling of Docs

Step 2: Create the Term-Document Matrix(TDM)

Machine learning models cannot read text directly; they require numbers. We convert the cleaned corpus into a mathematical format:

1. **Dictionary Mapping:** We assign a unique ID to every unique word found in the corpus.
2. **Bag-of-Words (BoW) Conversion:** We transform each document into a list of "Word ID: Frequency" pairs, which creates the TDM.

Step 3: Build the Model

1. Define the dimensions of the Latent Space: tell the algorithm how many "latent descriptions" to extract from the data distribution.
2. Use Latent Semantic Analysis/ Latent Dirichlet Allocation algorithms to cluster the words into this number of topics.

Case Study 3: Topic Modeling of Docs

Step 3: Build the Model

The core assumption for both LSA and LDA is that **"words that occur in similar contexts tend to have similar meanings"**.

While **LSA** is a mathematical shortcut that summarizes which words are related, **LDA** is a statistical guess that explains how those words were mixed into topics.

Feature	LSA (Latent Semantic Analysis)	LDA (Latent Dirichlet Allocation)
Focus on Meaning	Spatial Distance: Meaning is defined by how close words are located on a mathematical map.	Thematic Context: Meaning is defined by the likelihood of a word appearing in a specific subject.
Word Assignment	Fixed Weights: Assigns a permanent score (weight) to each word in a topic.	Fluid Probabilities: Assigns a percentage chance that a word belongs to a topic.
Logic Type	Deterministic: If you run it again on the same data, the results are identical.	Stochastic: Uses random sampling; results may shift slightly each time you run it.
Mathematical Engine	Linear Algebra	Bayesian Statistics
Output Values	Weights: Can be positive or negative.	Probabilities: Always positive, ranging from 0 to 1.
Best Used For	Small datasets.	Large, complex datasets.

Case Study 3: Topic Modeling of Docs

Step 3: Build the Model

LSA

Topic 1: $(0.555 \cdot \text{"water"} + 0.489 \cdot \text{"percent"} + 0.239 \cdot \text{"rain"})$

Topic 2: $(0.361 \cdot \text{"sleeping"} + 0.215 \cdot \text{"hour"} + 0.215 \cdot \text{"still"})$

Topic 3: $(0.562 \cdot \text{"water"} + -0.231 \cdot \text{"rain"} + -0.231 \cdot \text{"planet"})$

LDA

Topic 1: $(0.059 \cdot \text{"water"} + 0.055 \cdot \text{"percent"} + 0.021 \cdot \text{"rain"})$

Topic 2: $(0.078 \cdot \text{"water"} + 0.025 \cdot \text{"state"} + 0.025 \cdot \text{"solid"})$

Topic 3: $(0.040 \cdot \text{"sleeping"} + 0.024 \cdot \text{"expert"} + 0.024 \cdot \text{"life"})$

Case Study 3: Topic Modeling of Docs

Step 4: Determine which topics each doc falls into

LSA

Doc 1: [592.67, -30.96, -237.49]

Doc 2: [15.83, 531.27, -24.14]

Doc 3: [323.49, 27.4, 266.9]

Doc 4: [1.04, 101.83, -31.35]

Doc 5: [198.63, 4.89, 276.04]

LDA

Doc 1: [97.45, 1.41, 1.14]

Doc 2: [1.21, 1.21, 97.58]

Doc 3: [95.37, 2.36, 2.27]

Doc 4: [96.6, 1.69, 1.72]

Doc 5: [2.88, 94.32, 2.8]

Case Study 3: Topic Modeling of Docs

Step 5: Name the Topics

LSA

Topic 1: Global Hydrology

Topic 2: Sleep Behavior

Topic 3: Water Dynamics

LDA

Topic 1: Water Distribution

Topic 2: States of Water

Topic 3: Biological Sleep Cycles

Thank You

Join Group: <https://t.me/+NJ161vpFQ641ZDk0>